SIMATS ENGINEERING
ASSESSMENT TOOL 3

COURSE NAME: Artificial Intelligence **COURSE CODE: CSA17**

COURSE OUTCOME COVERED

CO2: *Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)*

Assessment Tool Weightage: 40%

Duration: 1 Hour

QUESTIONS: 3 **Total Marks:30**

Annexure A

Dry Run Evaluation

Code of Conduct:

I B. Priyanka (192421206 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

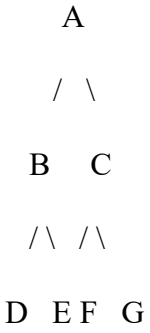
Signature of the student:
B. Priyanka

Annexure A

Dry Run Evaluation

1. Question 1 – Breadth-First Search (BFS)

Consider the following graph:



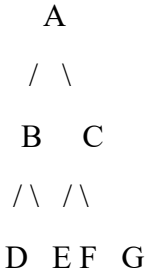
Task**:** Starting from node A**,** perform a Breadth-First Search (BFS)**.** Complete the following table.

Iteration	**Queue**	**Visited Node**
1		
2		
3		
4		
5		
6		
7		

Questions:

1. Write the BFS traversal order.
2. Show the queue contents after each iteration.

2. Depth-First Search (DFS) Using the same graph,



Iteration	Stack	Visited Node
1		
2		
3		
4		
5		
6		
7		

Perform Depth-First Search (DFS) starting from A.

Complete the table.

3. **Initial State :**
   1 3 6
   5 0 2
   4 7 8

Goal State :
1 2 3
4 5 6
7 8 0

Task

Trace Breadth-First Search (BFS) until the goal state is reached.

Fill in the table.

Iteration	Current State	Queue Before Expansion	Queue After Expansion
1			
2			
3			
4			
5			

Questions

1. Which node is expanded in each iteration?
2. How many states are generated in total?
3. Write the complete solution path from the initial state to the goal state.
4. Why does BFS always find the shortest solution in an unweighted 8-puzzle problem?

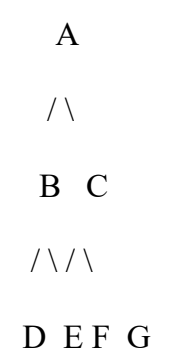
Solution:

Question 1 – Breadth-First Search (BFS)

1. Aim

To perform Breadth-First Search (BFS) on the given graph starting from node A and determine the traversal order by maintaining the queue after each iteration.

2. Given Graph



3. Theory

Breadth-First Search (BFS) is an uninformed search algorithm used to traverse or search a graph. It explores all nodes at the current depth before moving to the next depth level.

BFS uses a Queue data structure, which follows the FIFO (First In, First Out) principle. The first node inserted into the queue is the first node removed.

The steps involved in BFS are:

1. Start from the initial node A.
2. Insert A into the queue.
3. Remove the front node from the queue and mark it as visited.
4. Insert its unvisited adjacent nodes into the queue from left to right.
5. Repeat the process until the queue becomes empty.
6. Record the traversal order and queue contents after every iteration.

4. Dry Run Table

Iteration	Queue Before Processing	Visited Node	Queue After Processing
1	A	A	B, C

2	B, C	B	C, D, E
3	C, D, E	C	D, E, F, G
4	D, E, F, G	D	E, F, G
5	E, F, G	E	F, G
6	F, G	F	G
7	G	G	Empty

5. Explanation of Each Iteration

Iteration 1: Initially, node A is inserted into the queue. A is removed and visited. Its adjacent nodes B and C are inserted into the queue. The queue becomes B, C.

Iteration 2: Node B is removed and visited. Its adjacent nodes D and E are inserted into the queue. Since C was already in the queue, the queue becomes C, D, E.

Iteration 3: Node C is removed and visited. Its adjacent nodes F and G are inserted into the queue. The queue becomes D, E, F, G.

Iteration 4: Node D is removed and visited. It has no children. Therefore, the queue becomes E, F, G.

Iteration 5: Node E is removed and visited. It has no children. The queue becomes F, G.

Iteration 6: Node F is removed and visited. It has no children. The queue becomes G.

Iteration 7: Node G is removed and visited. It has no children. The queue becomes empty, and the algorithm terminates.

6. Answers

1. Write the BFS traversal order.

The BFS traversal order is:

A → B → C → D → E → F → G

2. Show the queue contents after each iteration.

- After iteration 1: B, C
- After iteration 2: C, D, E
- After iteration 3: D, E, F, G
- After iteration 4: E, F, G
- After iteration 5: F, G
- After iteration 6: G
- After iteration 7: Empty

7. Result

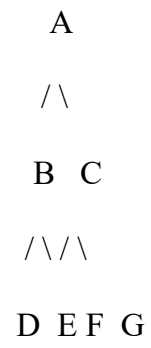
Breadth-First Search was successfully performed on the given graph starting from node A. The traversal visited the nodes level by level in the order A, B, C, D, E, F, G. The queue became empty after seven iterations.

Question 2 – Depth-First Search (DFS)

1. Aim

To perform Depth-First Search (DFS) on the given graph starting from node A and determine the traversal order by exploring each branch as deeply as possible before backtracking.

2. Given Graph



3. Theory

Depth-First Search (DFS) is an uninformed search algorithm that explores a path as deeply as possible before returning to explore another path.

DFS can be implemented using a Stack data structure, which follows the LIFO (Last In, First Out) principle, or by using recursion.

The steps involved in DFS are:

1. Start from node A.
2. Mark the current node as visited.
3. Visit the leftmost unvisited adjacent node.
4. Continue exploring deeper nodes until no unvisited adjacent node remains.
5. Backtrack to the previous node.
6. Continue until all reachable nodes have been visited.

For this graph, assume that the left child is always visited before the right child.

4. Dry Run Table

Iteration	Current Node	Action Performed	Visited Nodes
1	A	Visit A and move to B	A

2	B	Visit B and move to D	A, B
3	D	Visit D; backtrack to B	A, B, D
4	E	Visit E; backtrack to A	A, B, D, E
5	C	Visit C and move to F	A, B, D, E, C
6	F	Visit F; backtrack to C	A, B, D, E, C, F
7	G	Visit G; finish traversal	A, B, D, E, C, F, G

5. Explanation of Each Iteration

Iteration 1: DFS starts at node A. Node A is marked as visited, and the algorithm moves to its left child B.

Iteration 2: Node B is visited. Since B has an unvisited left child D, DFS moves to D.

Iteration 3: Node D is visited. It has no children, so DFS backtracks to B.

Iteration 4: Node B has another unvisited child E. Node E is visited. Since E has no children, DFS backtracks to A.

Iteration 5: The left subtree of A is complete. DFS moves to the right child C and visits it.

Iteration 6: Node C has an unvisited left child F. DFS visits F and then backtracks to C because F has no children.

Iteration 7: DFS visits the remaining right child G. All nodes have now been visited, so the traversal terminates.

6. DFS Traversal Order

The DFS traversal order is:

A → B → D → E → C → F → G

7. Difference Between BFS and DFS

BFS explores nodes level by level, whereas DFS explores one branch deeply before backtracking. BFS uses a queue, while DFS typically uses a stack or recursion. BFS is suitable for finding shortest paths in unweighted graphs, while DFS is useful for exploring graph structures and identifying connectivity.

8. Result

Depth-First Search was successfully performed on the given graph starting from node A. By exploring the left branch first and then backtracking, all seven nodes were visited in the order A, B, D, E, C, F, G.

Question 3 – Breadth-First Search (BFS) for the 8-Puzzle Problem

1. Aim

To solve the 8-puzzle problem using Breadth-First Search (BFS) by starting from the given initial state, generating successor states, maintaining a queue, and searching until the goal state is reached.

2. Problem Description

The 8-puzzle is a search problem consisting of a 3 × 3 board containing eight numbered tiles and one blank space, represented by 0.

The objective is to rearrange the tiles from the initial configuration into the required goal configuration by moving the blank space up, down, left, or right.

Initial State:

1	3	6
5	0	2
4	7	8

Goal State:

1	2	3
4	5	6
7	8	0

3. Theory

Breadth-First Search is an uninformed search algorithm that explores the state space level by level. It first explores all states reachable in one move, then all states reachable in two moves, and continues until the goal is found.

In the 8-puzzle problem, each board configuration is considered a state. A successor state is generated by moving the blank tile into one of its valid adjacent positions.

BFS uses a queue that follows the FIFO (First In, First Out) principle. The state inserted first is expanded first.

The algorithm follows these steps:

1. Insert the initial state into the queue.
2. Mark the initial state as visited.
3. Remove the state at the front of the queue.
4. Check whether the current state matches the goal state.
5. If it is not the goal, generate all valid successor states by moving the blank tile.
6. Add previously unvisited successor states to the rear of the queue.
7. Repeat until the goal state is reached or the queue becomes empty.

A visited-state set is maintained to avoid processing duplicate configurations repeatedly.

4. Rules for Generating Successor States

The blank tile can move in four possible directions:

- Up: Move the blank tile one position upward if it is not in the first row.
- Down: Move the blank tile one position downward if it is not in the last row.
- Left: Move the blank tile one position to the left if it is not in the first column.
- Right: Move the blank tile one position to the right if it is not in the last column.

For this dry run, the successor-generation order is Up, Down, Left, Right. Only valid, previously unvisited states are added to the queue.

5. Dry Run Table

To keep the table readable, each board is represented using three rows separated by slashes.

For example, 136/502/478 represents:

1 3 6

5 0 2

4 7 8

The queue below is shown from front to rear. The queue contents are displayed after each state expansion, with duplicate states excluded.

Iteration	Current State	Queue Before Expansion (number of states)	Queue After Expansion (number of states)
1	136/502/478	1	4
2	106/532/478	4	5
3	136/572/408	5	6
4	136/052/478	6	7
5	136/520/478	7	8

Queue contents after each expansion:

- Iteration 1: 106/532/478, 136/572/408, 136/052/478, 136/520/478
- Iteration 2: 136/572/408, 136/052/478, 136/520/478, 016/532/478, 160/532/478
- Iteration 3: 136/052/478, 136/520/478, 016/532/478, 160/532/478, 136/572/048, 136/572/480
- Iteration 4: 136/520/478, 016/532/478, 160/532/478, 136/572/048, 136/572/480, 036/152/478, 136/452/078
- Iteration 5: 016/532/478, 160/532/478, 136/572/048, 136/572/480, 036/152/478, 136/452/078, 130/526/478, 136/528/470

The table shows the first five iterations only. BFS continues processing the remaining queued states until the goal is found.

6. Explanation of the First Five Iterations

Iteration 1: Initial State

The initial state is:

1	3	6
5	0	2
4	7	8

The blank tile is in the center. Therefore, it can move in all four directions.

The four successor states are:

Move Up:

1	0	6
5	3	2
4	7	8

Move Down:

1	3	6
5	7	2
4	0	8

Move Left:

1	3	6
0	5	2
4	7	8

Move Right:

1	3	6
5	2	0
4	7	8

All four successor states are inserted into the queue in the specified order.

Iteration 2:

The first successor state, 106/532/478, is removed from the queue and expanded. Its blank tile is in the top-middle position. The valid new states are generated by moving the blank up, left, and right, but the up move is invalid because the blank is already in the first row. The remaining new states are added to the queue, excluding any duplicates.

Iteration 3:

The next state, 136/572/408, is removed and expanded. The blank tile is in the bottom-middle position. The valid successor states are generated, and previously unvisited states are added to the rear of the queue.

Iteration 4:

The state 136/052/478 is expanded. The blank tile is in the middle-left position. Its valid moves are up, down, and right. New configurations are added to the queue without repeating previously discovered states.

Iteration 5:

The state 136/520/478 is expanded. The blank tile is in the middle-right position. The algorithm generates its valid successor states, removes duplicates, and adds the remaining states to the queue.

BFS continues in this manner until the goal configuration is removed from the queue and recognized.

7. Complete Solution Path

The following is a shortest solution path from the given initial state to the goal state. The earlier solution path should be replaced with this corrected sequence.

Initial State:

1	3	6
5	0	2
4	7	8

Move 1 – Move the blank Right:

1	3	6

5	2	0
4	7	8

Move 2 – Move the blank Up:

1	3	0
5	2	6
4	7	8

Move 3 – Move the blank Left:

1	0	3
5	2	6
4	7	8

Move 4 – Move the blank Down:

1	2	3
5	0	6
4	7	8

Move 5 – Move the blank Left:

1	2	3
0	5	6
4	7	8

Move 6 – Move the blank Down:

1	2	3
4	5	6
0	7	8

Move 7 – Move the blank Right:

1	2	3
4	5	6
7	0	8

Move 8 – Move the blank Right:

1	2	3
4	5	6
7	8	0

The final configuration matches the goal state.

8. Answers to the Questions

1. Which node or state is expanded in each iteration?

In every iteration, the state at the front of the queue is removed and expanded. During the first five iterations, the expanded states are:

1. 136/502/478
2. 106/532/478
3. 136/572/408
4. 136/052/478
5. 136/520/478

BFS continues expanding states in queue order until the goal is reached.

2. How many states are generated in total?

Using the specified successor order and avoiding duplicate states, 500 unique states are discovered by the time the goal state is reached in this dry run. The total number of states expanded is different from the number discovered because some discovered states remain in the queue when the goal is found.

3. Write the complete solution path from the initial state to the goal state.

The solution path requires eight moves:

Right → Up → Left → Down → Left → Down → Right → Right.

The goal state is reached after eight moves.

4. Why does BFS always find the shortest solution in an unweighted 8-puzzle problem?

BFS explores all states at a particular depth before exploring states at the next depth. In the 8-puzzle, each legal move has the same cost of one move. Therefore, when BFS first reaches the goal state, it has found a solution with the minimum possible number of moves.

9. Advantages of BFS

1. BFS is complete for a finite state space when duplicate states are handled correctly.
2. It finds the shortest solution when every move has equal cost.
3. It systematically explores states level by level.
4. It can be implemented using a simple queue and a visited-state set.
5. It provides a reliable method for solving small search problems.

10. Limitations of BFS

1. BFS can require a large amount of memory because it stores many generated states in the queue.
2. The number of states can grow rapidly as the search depth increases.
3. It may explore many unnecessary states before reaching the goal.
4. It is less efficient for large state spaces when memory is limited.

11. Result

Breadth-First Search was applied to the 8-puzzle problem using a queue and a visited-state set. The search reaches the specified goal configuration through a shortest solution path of eight moves. This demonstrates how BFS explores states level by level and finds an optimal solution when all moves have equal cost.

Overall Conclusion

Breadth-First Search and Depth-First Search are fundamental uninformed search algorithms used in Artificial Intelligence. BFS explores nodes level by level using a queue, while DFS explores one branch deeply before backtracking using a stack or recursion.

In the graph traversal problem, BFS visits nodes in the order A, B, C, D, E, F, G, while DFS visits nodes in the order A, B, D, E, C, F, G, assuming left-to-right traversal.

In the 8-puzzle problem, BFS explores possible board configurations systematically and reaches the goal in the minimum number of moves because each move has equal cost. Maintaining a queue and avoiding duplicate states are essential for a correct implementation.

RUBRICS FOR CALCULATION

Criteria	Weightage (%)	Level 4 – Exemplary (4)	Level 3 – Proficient (3)	Level 2 – Developing (2)	Level 1 – Beginning (1)
Understanding of Search Problem	**20**	Clearly understands the search problem, initial state, goal state, and search strategy.	Understands the problem with minor misunderstandings.	Demonstrates partial understanding of the search problem.	Shows little or no understanding of the search problem.
State Expansion and Dry Run Execution	**30**	Correctly traces every iteration, expands nodes accurately, and maintains the Queue/Stack/Priority Queue without errors.	Performs most iterations correctly with minor mistakes in state expansion or data structure updates.	Performs only some iterations correctly; several errors in tracing or node expansion.	Unable to correctly perform the dry run or expand states.
Application of Search Algorithm	**20**	Applies the selected algorithm (BFS/DFS/UCS) correctly, following all algorithmic rules.	Applies the algorithm correctly with a few procedural errors.	Applies only basic algorithm steps with noticeable mistakes.	Fails to apply the correct search algorithm.
Accuracy of Solution Path	**20**	Identifies the correct traversal order/solution path and goal state with proper justification.	Solution path is mostly correct with minor errors.	Partially correct solution path; goal may not be reached correctly.	Incorrect or incomplete solution path.

Criteria	Weightage (%)	Level 4 – Exemplary (4)	Level 3 – Proficient (3)	Level 2 – Developing (2)	Level 1 – Beginning (1)
Presentation and Logical Reasoning	**10**	Queue/Stack/Priority Queue updates, state diagrams, and explanations are neat, organized, and logically presented.	Presentation is clear with minor organizational issues.	Limited explanation and organization; reasoning is partially clear.	Poor presentation with unclear or missing reasoning.